LEZIONE 3 - Sistemi Numerici e Rappresentazione Binaria degli Interi

Durata: 4 ore | Teoria: 2h | Esercizi: 2h

PARTE TEORICA (2 ore)

3.1 Sistemi di Numerazione Posizionali (25 min)

Cosa Significa "Posizionale"?

Un sistema di numerazione è **posizionale** quando il valore di una cifra dipende sia dal suo valore intrinseco che dalla sua **posizione**.

Esempio in base 10:
Il numero **352** significa:

- 3 centinaia (3 × 100)
- 5 decine (5 × 10)
- 2 unità (2 × 1)

Formula generale:

$$
\begin{aligned}
352_{10} &= 3\times10^2 + 5\times10^1 + 2\times10^0 \\
&= 3\times100 + 5\times10 + 2\times1 \\
&= 300 + 50 + 2 \\
&= 352
\end{aligned}
$$

Componenti di un Sistema Posizionale

1. Base (o radice):

- Determina quante cifre diverse si possono usare

- Base 10: cifre da 0 a 9
- Base 2: cifre 0 e 1
- Base 8: cifre da 0 a 7
- Base 16: cifre da 0 a 9 e A a F

2. Cifre:

- Simboli usati per rappresentare valori
- Il valore massimo di una cifra è (base - 1)

3. Peso Posizionale:

- Ogni posizione ha un peso che è una potenza della base
- Le posizioni si numerano da destra verso sinistra partendo da 0

Rappresentazione generale:

$$N = d_n \times base^n + d_{n-1} \times base^{n-1} + \ldots + d_1 \times base^1 + d_0 \times base^0$$

3.2 Il Sistema Decimale (Base 10) (10 min)

Il sistema **decimale** è quello che usiamo quotidianamente.

Caratteristiche:

- **Base**: 10
- **Cifre**: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- **Pesi posizionali**: …10^3, 10^2, 10^1, 10^0 = …1000, 100, 10, 1

Esempio Dettagliato: 2547_{10}

```
Posizione:   3    2    1    0
Cifra:       2    5    4    7
Peso:      10³  10²  10¹  10⁰
          1000  100   10    1

Valore: 2×1000 + 5×100 + 4×10 + 7×1
```

```
    = 2000 + 500 + 40 + 7
    = 2547
```

Numeri con Virgola:

```
Numero: 123.45₁₀

Posizione:  2   1   0  -1  -2
Cifra:      1   2   3   4   5
Peso:      10² 10¹ 10⁰ 10⁻¹ 10⁻²
          100  10   1  0.1  0.01

Valore: 1×100 + 2×10 + 3×1 + 4×0.1 + 5×0.01
    = 100 + 20 + 3 + 0.4 + 0.05
    = 123.45
```

3.3 Il Sistema Binario (Base 2) (30 min)

Il sistema **binario** è fondamentale per i computer, che lavorano solo con due stati: acceso/spento, vero/falso, 1/0.

Caratteristiche:

- **Base**: 2
- **Cifre**: 0, 1 (chiamate **bit** = binary digit)
- **Pesi posizionali**: …2^3, 2^2, 2^1, 2^0 = …8, 4, 2, 1

Tabella delle Potenze di 2 (da memorizzare!)

Esponente	Potenza	Valore Decimale
2^0	2^0	1
2^1	2^1	2

Esponente	Potenza	Valore Decimale
2^2	2^2	4
2^3	2^3	8
2^4	2^4	16
2^5	2^5	32
2^6	2^6	64
2^7	2^7	128
2^8	2^8	256
2^9	2^9	512
2^{10}	2^{10}	1024
2^{16}	2^{16}	65536

Esempio: Convertire 1011_2 in Decimale

Posizione: 3 2 1 0
Cifra: 1 0 1 1
Peso: 2^3 2^2 2^1 2^0
8 4 2 1

Valore: 1×8 + 0×4 + 1×2 + 1×1
= 8 + 0 + 2 + 1
= 11_{10}

Conversione Binario → Decimale (Metodo Generale)

Formula:

Somma di (cifra × 2^posizione) per ogni bit

Esempi:

1. 10101_2 in decimale:

$$
\begin{aligned}
&= 1\times2^4 + 0\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 \\
&= 1\times16 + 0\times8 + 1\times4 + 0\times2 + 1\times1 \\
&= 16 + 0 + 4 + 0 + 1 \\
&= 21_{10}
\end{aligned}
$$

1. 11111111_2 in decimale:

$$
\begin{aligned}
&= 1\times128 + 1\times64 + 1\times32 + 1\times16 + 1\times8 + 1\times4 + 1\times2 + 1\times1 \\
&= 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 \\
&= 255_{10}
\end{aligned}
$$

1. 10000000_2 in decimale:

$$
= 1\times128 = 128_{10}
$$

Conversione Decimale → Binario

Metodo delle Divisioni Successive:

1. Dividi il numero decimale per 2
2. Annota il resto (0 o 1)
3. Usa il quoziente per la divisione successiva
4. Ripeti fino a quando il quoziente diventa 0
5. Il numero binario si legge dai resti **dal basso verso l'alto**

Esempio 1: Convertire 13_{10} in binario

```
13 ÷ 2 = 6  resto 1  ←┐
 6 ÷ 2 = 3  resto 0   |
 3 ÷ 2 = 1  resto 1   | Leggi dal basso
 1 ÷ 2 = 0  resto 1  ←┘ verso l'alto
```

Risultato: $13_{10} = 1101_2$

Verifica:

$1101_2 = 1\times8 + 1\times4 + 0\times2 + 1\times1 = 8+4+0+1 = 13_{10}$ ✓

Esempio 2: Convertire 25_{10} in binario

```
25 ÷ 2 = 12  resto 1  ←┐
12 ÷ 2 = 6   resto 0   |
 6 ÷ 2 = 3   resto 0   | Leggi
 3 ÷ 2 = 1   resto 1   |
 1 ÷ 2 = 0   resto 1  ←┘
```

Risultato: $25_{10} = 11001_2$

Esempio 3: Convertire 100_{10} in binario

```
100 ÷ 2 = 50  resto 0  ←┐
 50 ÷ 2 = 25  resto 0   |
 25 ÷ 2 = 12  resto 1   |
 12 ÷ 2 = 6   resto 0   |
  6 ÷ 2 = 3   resto 0   | Leggi
  3 ÷ 2 = 1   resto 1   |
  1 ÷ 2 = 0   resto 1  ←┘
```

Risultato: $100_{10} = 1100100_2$

Operazioni Aritmetiche in Binario

Addizione:
Regole base:

- 0 + 0 = 0
- 0 + 1 = 1
- 1 + 0 = 1
- 1 + 1 = 10 (scrivi 0, riporta 1)
- 1 + 1 + 1 (riporto) = 11 (scrivi 1, riporta 1)

Esempio: $1011_2 + 0110_2$

```
 1011  (11₁₀)
+ 0110  ( 6₁₀)
------
 10001  (17₁₀)

Passo per passo:
Colonna 0: 1+0 = 1
Colonna 1: 1+1 = 0, riporto 1
Colonna 2: 0+1+1(riporto) = 0, riporto 1
Colonna 3: 1+0+1(riporto) = 0, riporto 1
Colonna 4: 1(riporto) = 1
```

Sottrazione:
Regole base:

- 0 - 0 = 0
- 1 - 0 = 1
- 1 - 1 = 0
- 0 - 1 = 1 (con prestito dalla posizione successiva)

Esempio: $1010_2 - 0011_2$

```
 1010  (10₁₀)
- 0011  ( 3₁₀)
```

```
------
 0111  (7₁₀)
```

3.4 Sistema Ottale (Base 8) (15 min)

Il sistema **ottale** usa 8 cifre ed era molto utilizzato nei primi computer.

Caratteristiche:

- **Base**: 8
- **Cifre**: 0, 1, 2, 3, 4, 5, 6, 7
- **Pesi posizionali**: …8^3, 8^2, 8^1, 8^0 = …512, 64, 8, 1

Conversione Ottale → Decimale

Esempio: 347_8 in decimale

$$
\begin{aligned}
347_8 &= 3\times8^2 + 4\times8^1 + 7\times8^0 \\
&= 3\times64 + 4\times8 + 7\times1 \\
&= 192 + 32 + 7 \\
&= 231_{10}
\end{aligned}
$$

Conversione Decimale → Ottale

Metodo: Divisioni successive per 8

Esempio: 100_{10} in ottale

```
100 ÷ 8 = 12  resto 4  ←┐
 12 ÷ 8 = 1   resto 4   | Leggi
  1 ÷ 8 = 0   resto 1  ←┘

Risultato: 100₁₀ = 144₈
```

Conversione Binario ↔ Ottale

Regola veloce: Ogni cifra ottale corrisponde a **3 bit**.

Tabella di conversione:

Ottale	Binario
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Esempio 1: 101110101_2 → Ottale

Raggruppa da destra in gruppi di 3:
101 110 101
 5 6 5

Risultato: $101110101_2 = 565_8$

Esempio 2: 743_8 → Binario

7 4 3
111 100 011

Risultato: $743_8 = 111100011_2$

3.5 Sistema Esadecimale (Base 16) (20 min)

Il sistema **esadecimale** è molto usato in informatica per rappresentare indirizzi di memoria e codici colore.

Caratteristiche:

- **Base**: 16
- **Cifre**: 0-9, A-F (dove A=10, B=11, C=12, D=13, E=14, F=15)
- **Pesi posizionali**: …16^3, 16^2, 16^1, 16^0 = …4096, 256, 16, 1

Tabella Esadecimale

Esadecimale	Decimale	Binario
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101

Esadecimale	Decimale	Binario
E	14	1110
F	15	1111

Conversione Esadecimale → Decimale

Esempio: $2A3_{16}$ in decimale

$$
\begin{aligned}
2A3_{16} &= 2\times16^2 + A\times16^1 + 3\times16^0 \\
&= 2\times256 + 10\times16 + 3\times1 \\
&= 512 + 160 + 3 \\
&= 675_{10}
\end{aligned}
$$

Esempio: FF_{16} in decimale

$$
\begin{aligned}
FF_{16} &= F\times16^1 + F\times16^0 \\
&= 15\times16 + 15\times1 \\
&= 240 + 15 \\
&= 255_{10}
\end{aligned}
$$

Conversione Decimale → Esadecimale

Metodo: Divisioni successive per 16

Esempio: 1000_{10} in esadecimale

```
1000 ÷ 16 = 62  resto 8   ←┐
 62 ÷ 16 = 3   resto 14(E) | Leggi
  3 ÷ 16 = 0   resto 3   ←┘

Risultato: 1000₁₀ = 3E8₁₆
```

Conversione Binario ↔ Esadecimale

Regola veloce: Ogni cifra esadecimale corrisponde a **4 bit**.

Esempio 1: 11010110_2 → Esadecimale

Raggruppa da destra in gruppi di 4:
1101 0110
D 6

Risultato: $11010110_2 = D6_{16}$

Esempio 2: $1A7F_{16}$ → Binario

1 A 7 F
0001 1010 0111 1111

Risultato: $1A7F_{16} = 0001101001111111_2$

Esempio 3: Semplifica 101111010011_2

Conversione diretta: lungo e complicato

Via esadecimale:
1011 1101 0011
B D 3

Più facile ricordare $BD3_{16}$!

3.6 Rappresentazione degli Interi Positivi (20 min)

Bit, Byte e Parole

Terminologia:

- **Bit**: singola cifra binaria (0 o 1)
- **Nibble**: 4 bit (una cifra esadecimale)
- **Byte**: 8 bit (standard de facto)
- **Word (Parola)**: dimensione dipende dall'architettura (16, 32, 64 bit)

Range di Rappresentazione

Con **n bit** si possono rappresentare **2^n** numeri diversi.

Per interi **positivi** (senza segno):

- Valore minimo: 0
- Valore massimo: $2^n - 1$

Tabella Range:

Bit	Numeri Rappresentabili	Range (0 a max)
1 bit	2	0 - 1
2 bit	4	0 - 3
3 bit	8	0 - 7
4 bit	16	0 - 15
8 bit	256	0 - 255
16 bit	65,536	0 - 65,535
32 bit	4,294,967,296	0 - 4,294,967,295
64 bit	18,446,744,073,709,551,616	0 - 18,446,744,073,709,551,615

Esempi:

8 bit senza segno:

$$\text{Minimo: } 00000000_2 = 0_{10}$$
$$\text{Massimo: } 11111111_2 = 255_{10}$$

16 bit senza segno:

$$\text{Minimo: } 0000000000000000_2 = 0_{10}$$
$$\text{Massimo: } 1111111111111111_2 = 65535_{10}$$

Tipi in C (Unsigned)

In C, i tipi senza segno si dichiarano con `unsigned`:

```
unsigned char c;      // 8 bit: 0 a 255
unsigned short s;     // 16 bit: 0 a 65535
unsigned int i;       // 32 bit: 0 a 4294967295
unsigned long l;      // 32/64 bit (dipende da architettura)
unsigned long long ll; // 64 bit
```

3.7 Rappresentazione degli Interi con Segno (20 min)

Problema: Come Rappresentare i Negativi?

Con n bit dobbiamo rappresentare sia positivi che negativi. Servono 3 metodi storici:

1. Modulo e Segno (Sign-Magnitude)

- **Bit più significativo (MSB)**: 0 = positivo, 1 = negativo
- **Altri bit**: valore assoluto

Con 8 bit:

```
+5 = 00000101
-5 = 10000101

+0 = 00000000
-0 = 10000000  (problema: due rappresentazioni per zero!)
```

Svantaggi:

- Due rappresentazioni per lo zero
- Circuiti di addizione complessi
- Non più usato nei computer moderni

2. Complemento a 1

- **Positivi**: come al solito
- **Negativi**: inverti tutti i bit del corrispondente positivo

Con 8 bit:

```
+5 = 00000101
-5 = 11111010 (NOT di 00000101)

+0 = 00000000
-0 = 11111111  (problema: ancora due zeri!)
```

Svantaggio: Ancora due rappresentazioni per zero

3. Complemento a 2 (Usato Oggi!)

Metodo più importante! Tutti i computer moderni usano il complemento a 2.

Regola per calcolare il complemento a 2:

1. Prendi il valore binario positivo
2. Inverti tutti i bit (complemento a 1)
3. Aggiungi 1

Esempio: -5 in complemento a 2 (8 bit)

```
+5 = 00000101
Complemento a 1: 11111010
Aggiungi 1:     + 00000001
                -----------
-5 = 11111011
```

Esempio: -1 in complemento a 2 (8 bit)

```
+1 = 00000001
Complemento a 1: 11111110
Aggiungi 1:     + 00000001
                -----------
-1 = 11111111
```

Esempio: -128 in complemento a 2 (8 bit)

```
+128 non rappresentabile in 8 bit signed!
Ma -128 = 10000000
```

Range con Complemento a 2

Con **n bit** in complemento a 2:

- Valore minimo: -2^{n-1}
- Valore massimo: $+2^{n-1} - 1$

Tabella Range Signed:

Bit	Range (min a max)
8 bit	-128 a +127
16 bit	-32,768 a +32,767
32 bit	-2,147,483,648 a +2,147,483,647

Bit	Range (min a max)
64 bit	-9,223,372,036,854,775,808 a +9,223,372,036,854,775,807

Nota: C'è un numero negativo in più del massimo positivo!

Come Riconoscere il Segno

Regola: Se il bit più significativo (MSB) è:

- **0** → numero positivo
- **1** → numero negativo

Esempi 8 bit:

```
00000101 = +5  (MSB = 0)
10000101 = -123 (MSB = 1)
11111111 = -1  (MSB = 1)
01111111 = +127 (MSB = 0)
```

Convertire da Complemento a 2 a Decimale

Se MSB = 0 (positivo): conversione normale

Se MSB = 1 (negativo):

1. Sottrai 1
2. Inverti tutti i bit
3. Converti in decimale
4. Aggiungi il segno meno

Esempio: 11111011_2 (8 bit) in decimale

```
MSB = 1, quindi negativo

Sottrai 1:      11111011
              -00000001
              ---------
```

```
              11111010

Inverti bit:    00000101

Converti:       00000101 = 5₁₀

Risultato: -5₁₀
```

Metodo alternativo veloce:

$$11111011_2 = -128 + 64 + 32 + 16 + 8 + 2 + 1$$
$$= -128 + 123$$
$$= -5_{10}$$

Il bit MSB vale $-2^7 = -128$!

Vantaggi del Complemento a 2

1. **Una sola rappresentazione per zero**
2. **Addizione semplice**: stessa circuiteria per positivi e negativi
3. **Estensione del segno**: facile convertire tra dimensioni diverse

Esempio Addizione:

```
 +5:  00000101
+ -3:  11111101
    ---------
 +2:  00000010  (ignora il riporto finale)
```

Overflow e Underflow

Overflow: il risultato supera il massimo rappresentabile

Esempio 8 bit signed:

```
 +100:  01100100
+ +50:   00110010
     ----------
 -106:  10010110  ← OVERFLOW! Risultato errato
```

Underflow: il risultato è minore del minimo rappresentabile

```
 -100:  10011100
+ -50:   11001110
     ----------
 +106:  01101010  ← UNDERFLOW! Risultato errato
```

Come rilevare overflow in somma:

- Due positivi → risultato negativo = OVERFLOW
- Due negativi → risultato positivo = UNDERFLOW
- Positivo + negativo → NO overflow possibile

PARTE PRATICA (2 ore)

ESERCIZI GUIDATI

Esercizio 1: Conversioni Binario ↔ Decimale (20 min)

1.1 Converti in decimale i seguenti numeri binari:

1. 1010_2

```
Soluzione:
1010₂ = 1×2³ + 0×2² + 1×2¹ + 0×2⁰
      = 8 + 0 + 2 + 0
      = 10₁₀
```

1. 11001_2

Soluzione:
$11001_2 = 1\times16 + 1\times8 + 0\times4 + 0\times2 + 1\times1$
$= 16 + 8 + 1$
$= 25_{10}$

1. 10000001_2

Soluzione:
$10000001_2 = 1\times128 + 0\times64 + \ldots + 1\times1$
$= 128 + 1$
$= 129_{10}$

1. 11111111_2

Soluzione:
$11111111_2 = 128+64+32+16+8+4+2+1$
$= 255_{10}$
(oppure: 2^8-1 = 256-1 = 255)

1.2 Converti in binario i seguenti numeri decimali:

1. 18_{10}

Soluzione:
18 ÷ 2 = 9 resto 0 ←┐
9 ÷ 2 = 4 resto 1 |
4 ÷ 2 = 2 resto 0 |
2 ÷ 2 = 1 resto 0 |
1 ÷ 2 = 0 resto 1 ←┘

$18_{10} = 10010_2$

1. 63_{10}

Soluzione:
63 ÷ 2 = 31 resto 1 ←┐
31 ÷ 2 = 15 resto 1 |
15 ÷ 2 = 7 resto 1 |
7 ÷ 2 = 3 resto 1 |
3 ÷ 2 = 1 resto 1 |
1 ÷ 2 = 0 resto 1 ←┘

$63_{10} = 111111_2$

1. 200_{10}

Soluzione:
200 ÷ 2 = 100 resto 0
100 ÷ 2 = 50 resto 0
50 ÷ 2 = 25 resto 0
25 ÷ 2 = 12 resto 1
12 ÷ 2 = 6 resto 0
6 ÷ 2 = 3 resto 0
3 ÷ 2 = 1 resto 1
1 ÷ 2 = 0 resto 1

$200_{10} = 11001000_2$

Esercizio 3: Operazioni Aritmetiche in Binario (25 min)

3.1 Esegui le seguenti addizioni in binario:

1. $1011_2 + 0110_2$

Soluzione:
1011 (11_{10})
+ 0110 (6_{10})

```
------
 10001  (17₁₀)
```

Verifica: 11 + 6 = 17 ✓

1. $1111_2 + 0001_2$

Soluzione:

```
  1111  (15₁₀)
+ 0001  ( 1₁₀)
------
 10000  (16₁₀)
```

Verifica: 15 + 1 = 16 ✓

1. $10110_2 + 1101_2$

Soluzione:

```
  10110  (22₁₀)
+  1101  (13₁₀)
-------
 100011  (35₁₀)
```

Passo per passo:

```
 1 1 1   (riporti)
  10110
+  1101
-------
 100011
```

Verifica: 22 + 13 = 35 ✓

3.2 Esegui le seguenti sottrazioni in binario:

1. $1010_2 - 0011_2$

Soluzione:

$$
\begin{array}{r}
1010 \ (10_{10}) \\
- \ 0011 \ (3_{10}) \\
\hline
0111 \ (7_{10})
\end{array}
$$

Verifica: 10 - 3 = 7 ✓

1. $10000_2 - 0001_2$

Soluzione:

$$
\begin{array}{r}
10000 \ (16_{10}) \\
- \ 0001 \ (1_{10}) \\
\hline
01111 \ (15_{10})
\end{array}
$$

Verifica: 16 - 1 = 15 ✓

3.3 Moltiplica $101_2 \times 11_2$:

Soluzione:

$$
\begin{array}{r}
101 \ (5_{10}) \\
\times \ 11 \ (3_{10}) \\
\hline
101 \ (5\times1) \\
101 \ (5\times1, \text{spostato}) \\
\hline
1111 \ (15_{10})
\end{array}
$$

Verifica: 5 × 3 = 15 ✓

Esercizio 4: Complemento a 2 (30 min)

4.1 Calcola il complemento a 2 dei seguenti numeri (8 bit):

1. Complemento a 2 di `+10`:

```
Soluzione:
+10 = 00001010

Step 1 - Inverti i bit:
11110101

Step 2 - Aggiungi 1:
  11110101
+ 00000001
----------
  11110110

-10 in complemento a 2 = 11110110₂
```

1. Complemento a 2 di `+25`:

```
Soluzione:
+25 = 00011001

Step 1 - Inverti i bit:
11100110

Step 2 - Aggiungi 1:
  11100110
+ 00000001
----------
  11100111
```

-25 in complemento a 2 = 11100111_2

1. Complemento a 2 di `+127`:

Soluzione:
+127 = 01111111

Step 1 - Inverti i bit:
10000000

Step 2 - Aggiungi 1:
  10000000
+ 00000001

  10000001

-127 in complemento a 2 = 10000001_2

1. Complemento a 2 di `+1`:

Soluzione:
+1 = 00000001

Step 1 - Inverti i bit:
11111110

Step 2 - Aggiungi 1:
  11111110
+ 00000001

  11111111

-1 in complemento a 2 = 11111111_2

4.2 Converti da complemento a 2 (8 bit) a decimale:

1. 10000101_2

Soluzione:
MSB = 1, quindi negativo

Metodo 1 (tradizionale):
Step 1 - Sottrai 1:
  10000101
- 00000001

  10000100

Step 2 - Inverti bit:
01111011

Step 3 - Converti:
01111011 = 64+32+16+8+2+1 = 123_{10}

Step 4 - Aggiungi segno:
Risultato: -123_{10}

Metodo 2 (veloce):
10000101_2 = -128 + 0 + 0 + 0 + 0 + 4 + 0 + 1
= -128 + 5
= -123_{10}

1. 11111111_2

Soluzione:
MSB = 1, quindi negativo

Metodo veloce:
$11111111_2 = -128 + 127 = -1_{10}$

(Oppure: tutti 1 in complemento a 2 = sempre -1)

1. 10000000_2

Soluzione:
MSB = 1, quindi negativo

$10000000_2 = -128 + 0 = -128_{10}$

(Il numero negativo più piccolo in 8 bit!)

1. 01111111_2

Soluzione:
MSB = 0, quindi positivo

$01111111_2 = 64+32+16+8+4+2+1 = 127_{10}$

(Il numero positivo più grande in 8 bit!)

4.3 Esegui le seguenti operazioni in complemento a 2 (8 bit):

1. $+5 + (-3)$

Soluzione:
+5 = 00000101
-3: complemento a 2 di +3
+3 = 00000011

Inverti: 11111100
+1: 11111101
-3 = 11111101

Addizione:
  00000101 (+5)
+ 11111101 (-3)

 100000010

Ignora il riporto (overflow oltre 8 bit):
00000010 = $+2_{10}$

Verifica: 5 + (-3) = 2 ✓

1. (-10) + (-20)

Soluzione:
+10 = 00001010
-10 = 11110110 (complemento a 2)

+20 = 00010100
-20 = 11101100 (complemento a 2)

Addizione:
  11110110 (-10)
+ 11101100 (-20)

 111100010

Ignora riporto oltre 8 bit:
11100010

Converti in decimale (è negativo):

11100010 = -128 + 96 + 32 + 2
= -128 + 130
= -30_{10} (WAIT! Errato!)

Metodo corretto:
11100010
- 1 = 11100001
NOT = 00011110 = 30_{10}
Quindi: -30_{10}

Verifica: -10 + (-20) = -30 ✓

Esercizio 5: Range e Overflow (20 min)

5.1 Determina il range di rappresentazione per:

1. 4 bit unsigned

Soluzione:
2^4 = 16 numeri
Range: 0 a 15
Minimo: $0000_2 = 0_{10}$
Massimo: $1111_2 = 15_{10}$

1. 4 bit signed (complemento a 2)

Soluzione:
Range: -2^3 a 2^3-1
Range: -8 a +7

Minimo: $1000_2 = -8_{10}$
Massimo: $0111_2 = +7_{10}$

1. 12 bit unsigned

Soluzione:
2^{12} = 4096 numeri
Range: 0 a 4095_{10}

1. 12 bit signed (complemento a 2)

Soluzione:
Range: -2^{11} a $2^{11}-1$
Range: -2048 a $+2047_{10}$

5.2 Identifica gli overflow (8 bit signed):

1. `+100 + +50`

Soluzione:
+100 = 01100100
+50 = 00110010

Addizione:
```
  01100100
+ 00110010
----------
  10010110
```

Risultato: 10010110_2
MSB = 1, quindi sembra negativo
Convertiamo: -106_{10}

OVERFLOW! ✗
Due positivi danno negativo = overflow
Risultato corretto sarebbe +150, ma non rappresentabile in 8 bit signed (max +127)

1. `+50 + (-30)`

Soluzione:
+50 = 00110010
-30 = 11100010 (complemento a 2 di +30)

Addizione:
```
  00110010
+ 11100010
----------
 100010100
```

Ignora riporto:
$00010100 = +20_{10}$

NESSUN OVERFLOW ✓
Verifica: 50 + (-30) = 20 ✓

1. (-100) + (-50)

Soluzione:
-100 = 10011100
-50 = 11001110

Addizione:
```
  10011100
+ 11001110
----------
 101101010
```

Ignora riporto:
$01101010 = +106_{10}$

UNDERFLOW! ✗

Due negativi danno positivo = underflow
Risultato corretto sarebbe -150, ma non rappresentabile

5.3 Qual è il valore di 11111111_2 in:

1. 8 bit unsigned

Soluzione:
$11111111_2 = 255_{10}$

1. 8 bit signed (complemento a 2)

Soluzione:
MSB = 1, quindi negativo
$11111111_2 = -1_{10}$

Esercizio 6: Problemi Applicativi (25 min)

6.1 Un sensore di temperatura invia valori in formato binario a 8 bit unsigned. Converti le seguenti letture:

1. 01100100_2 (temperatura in gradi Celsius)

Soluzione:
$01100100_2 = 64 + 32 + 4 = 100_{10}$
Temperatura: 100°C

1. 11111111_2 (temperatura massima rilevabile)

Soluzione:
$11111111_2 = 255_{10}$
Temperatura massima: 255°C

1. Se il sensore deve misurare temperature da -50°C a +100°C, è sufficiente 8 bit unsigned?

Soluzione:
NO! 8 bit unsigned va da 0 a 255.
Non può rappresentare valori negativi.

Serve 8 bit SIGNED (complemento a 2):
Range: -128 a +127
Copre da -50 a +100 ✓

6.2 Un sistema di controllo accessi usa codici a 16 bit. Quanti codici diversi sono possibili?

Soluzione:
Con 16 bit: $2^{16} = 65,536$ codici diversi

6.3 In un videogioco, le coordinate X,Y dei personaggi sono memorizzate come interi a 16 bit signed. Qual è l'area di gioco massima?

Soluzione:
16 bit signed: da -32,768 a +32,767

Coordinate X: da -32,768 a +32,767 (65,536 posizioni)
Coordinate Y: da -32,768 a +32,767 (65,536 posizioni)

Area totale: 65,536 × 65,536 = 4,294,967,296 posizioni

6.4 Un colore RGB usa 8 bit per canale (Rosso, Verde, Blu). Quanti colori diversi si possono rappresentare?

Soluzione:
Ogni canale: $2^8 = 256$ livelli (0-255)
Totale combinazioni: $256 \times 256 \times 256 = 16{,}777{,}216$ colori

Esempi:
Rosso puro: (255, 0, 0) = $FF0000_{16}$
Verde puro: (0, 255, 0) = $00FF00_{16}$
Blu puro: (0, 0, 255) = $0000FF_{16}$
Bianco: (255, 255, 255) = $FFFFFF_{16}$
Nero: (0, 0, 0) = 000000_{16}

6.5 Un contatore a 8 bit parte da 250 e viene incrementato di 1 per 10 volte. Cosa succede?

1. Se è unsigned:

Soluzione:
$250_{10} = 11111010_2$

Incrementi:
250 → 251 → 252 → 253 → 254 → 255 → 0 → 1 → 2 → 3 → 4

Dopo 10 incrementi: 4_{10}
(Overflow al 6° incremento: 255 + 1 = 0)

1. Se è signed (complemento a 2):

Soluzione:
250 non è rappresentabile in 8 bit signed (max +127)
Ma se partiamo da -6 (= 11111010 in complemento a 2):

-6 → -5 → -4 → -3 → -2 → -1 → 0 → 1 → 2 → 3 → 4

Dopo 10 incrementi: $+4_{10}$

ESERCIZI AUTONOMI DA COMPLETARE IN CLASSE

Esercizio 7: Tabella di Conversione

Completa la seguente tabella di conversione:

Decimale	Binario (8 bit)	Ottale	Esadecimale
15	00001111	17	F
32	?	?	?
?	01010101	?	?
?	?	144	?
200	?	?	?
?	?	?	A5

Soluzioni:

Decimale	Binario (8 bit)	Ottale	Esadecimale
15	00001111	17	F
32	00100000	40	20
85	01010101	125	55
100	01100100	144	64
200	11001000	310	C8
165	10100101	245	A5

Esercizio 8: Problemi Misti

8.1 Converti 10101010_2 in tutte le basi:

- Decimale (unsigned): _____
- Decimale (signed): _____
- Ottale: _____
- Esadecimale: _____

8.2 Trova il numero binario a 8 bit che rappresenta:

- Il più grande numero positivo in signed: _____
- Il più piccolo numero negativo in signed: _____
- Il numero 100 in unsigned: _____

8.3 Esegui $11110000_2 + 00001111_2$ e interpreta il risultato come:

- Unsigned: _____
- Signed (complemento a 2): _____

ESERCIZI DA SVOLGERE A CASA

Homework 1: Crea una tabella con tutti i numeri da 0 a 31 in:

- Decimale
- Binario (5 bit)
- Ottale
- Esadecimale

Homework 2: Calcola il complemento a 2 (8 bit) di tutti i numeri da -10 a +10 e verifica sommando numero + complemento che ottieni sempre zero (ignorando il riporto).

Homework 3: Scrivi un algoritmo (flow chart o pseudocodice) per convertire un numero decimale in binario usando il metodo delle divisioni successive.

Homework 4: Ricerca su internet e spiega brevemente:

- Cos'è la codifica BCD (Binary Coded Decimal)
- Cos'è la codifica Gray (o codifica riflessa)
- Dove vengono utilizzate queste codifiche

Homework 5: Problemi applicativi:
a) Quanti bit servono per rappresentare gli anni dal 1900 al 2100?
b) Un disco rigido ha capacità di 512 GB. Quanti bit servono per indirizzare ogni byte?
c) IPv4 usa indirizzi a 32 bit. Quanti indirizzi IP diversi esistono?

APPROFONDIMENTI

Storia dei Sistemi di Numerazione

Sistema Babilonese (2000 a.C.)

- Base 60 (sessagesimale)
- Ancora usato oggi: 60 secondi, 60 minuti, 360 gradi

Sistema Romano

- Non posizionale: VII, XIV, MCMXCIV
- Difficile fare calcoli!

Sistema Indiano-Arabo (500 d.C.)

- Base 10 posizionale
- Introdotto in Europa da Fibonacci (1202)

Sistema Binario

- Gottfried Leibniz (1679): “De Progressione Dyadica”
- George Boole (1854): algebra booleana
- Claude Shannon (1937): applicazione ai circuiti elettrici
- ENIAC (1945): primo computer elettronico binario

Perché i Computer Usano il Binario?

Ragioni tecnologiche:

1. **Affidabilità**: Più facile distinguere tra 2 stati (0/1) che tra 10 (0-9)
2. **Componenti semplici**: Transistor = interruttore on/off
3. **Rumore**: Sistema binario molto resistente ai disturbi elettrici
4. **Logica**: Algebra booleana (AND, OR, NOT) naturalmente binaria

Furono tentate alternative:

- **Computer ternari** (URSS, anni ’50): Setun usava base 3
- **Computer decimali**: ENIAC inizialmente era parzialmente decimale
- Tutti abbandonati perché il binario è più efficiente

Altre Rappresentazioni dei Negativi

Eccesso-K (Biased Representation):
Usato negli esponenti dei floating-point (IEEE 754)

- Aggiungi una costante K a tutti i numeri
- Esempio: Eccesso-128 con 8 bit
  - $0_{10} \rightarrow 128_{10} \rightarrow 10000000_2$
  - $-128_{10} \rightarrow 0_{10} \rightarrow 00000000_2$
  - $+127_{10} \rightarrow 255_{10} \rightarrow 11111111_2$

Il Problema dell’Anno 2038

Y2K Bug per Unix!

I sistemi Unix memorizzano il tempo come:

- Secondi dal 1 gennaio 1970
- Tipo `time_t`: intero signed a 32 bit

Range:

- Minimo: -2,147,483,648 sec → 13 dicembre 1901
- Massimo: +2,147,483,647 sec → **19 gennaio 2038 03:14:07 UTC**

Dopo questa data: overflow! Il tempo tornerebbe al 1901.

Soluzione: Passare a `time_t` a 64 bit (già implementato in sistemi moderni)

Curiosità: Base64

NON è base 64 matematica! È una codifica per trasmettere dati binari come testo.

Usa 64 caratteri:

- A-Z (26)
- a-z (26)
- 0-9 (10)
- ◦ e / (2)

Usato in:

- Email (MIME)
- URL
- Immagini embedded in HTML

Esempio:

```
"Hello" → "SGVsbG8="
```

Codici di Correzione Errori

Codice di Hamming:
Aggiunge bit di parità per rilevare e correggere errori

Esempio: trasmissione dati

```
Dati: 1011
Hamming: 1011010 (con 3 bit di parità)
```

Se 1 bit viene corrotto durante la trasmissione, può essere corretto automaticamente!

Usato in:

- RAM ECC (Error Correcting Code)
- Hard disk
- Trasmissioni satellitari

RIEPILOGO FORMULE

Conversione da Base B a Decimale

$$N_B = d_n \times B^n + d_{n-1} \times B^{n-1} + \dots + d_1 \times B^1 + d_0 \times B^0$$

Conversione da Decimale a Base B

Divisioni successive per B, leggi i resti dal basso verso l'alto

Range Unsigned (n bit)

Min = 0
Max = $2^n - 1$
Numeri = 2^n

Range Signed Complemento a 2 (n bit)

Min = -2^{n-1}
Max = $2^{n-1} - 1$
Numeri = 2^n

Complemento a 2

-X = NOT(X) + 1
oppure
$-X = 2^n - X$

Conversione Binario ↔ Ottale

1 cifra ottale = 3 bit

Conversione Binario ↔ Esadecimale

1 cifra esadecimale = 4 bit = 1 nibble

TABELLE DI RIFERIMENTO RAPIDO

Potenze di 2 (Memorizza!)

$2^0 = 1$
$2^1 = 2$
$2^2 = 4$
$2^3 = 8$
$2^4 = 16$
$2^5 = 32$
$2^6 = 64$
$2^7 = 128$
$2^8 = 256$
$2^9 = 512$
$2^{10} = 1{,}024$ (≈ 1K)
$2^{20} = 1{,}048{,}576$ (≈ 1M)
$2^{30} = 1{,}073{,}741{,}824$ (≈ 1G)

Prefissi Binari (IEC Standard)

1 KiB = 2^{10} byte = 1,024 byte
1 MiB = 2^{20} byte = 1,048,576 byte
1 GiB = 2^{30} byte ≈ 1.07 GB
1 TiB = 2^{40} byte ≈ 1.10 TB

Primi 16 Numeri in Tutte le Basi

Dec	Bin	Oct	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

PUNTI CHIAVE DELLA LEZIONE

✓ I computer usano il **sistema binario** (base 2) internamente

✓ **Ottale** (base 8) e **esadecimale** (base 16) semplificano la lettura del binario

✓ Con **n bit** si rappresentano $\mathbf{2^n}$ numeri diversi
✓ Unsigned: da **0** a $\mathbf{2^n-1}$
✓ Signed (complemento a 2): da $\mathbf{-2^{n-1}}$ a $\mathbf{2^{n-1}-1}$
✓ Il **complemento a 2** è lo standard per i numeri negativi
✓ **Overflow** = risultato troppo grande per il numero di bit
✓ **MSB** = Most Significant Bit = bit del segno in complemento a 2
✓ Esadecimale è fondamentale in programmazione (indirizzi, colori, debug)

PREPARAZIONE PER LA PROSSIMA LEZIONE

Nella prossima lezione studieremo:

- **Numeri decimali in binario** (virgola fissa e mobile)
- **Standard IEEE 754** (floating-point)
- **Rappresentazione dei caratteri**: ASCII
- **Charset estesi**: Unicode e UTF-8
- **Differenza tra numero e carattere**

Porta con te:

- Esercizi homework completati
- Calcolatrice scientifica
- Tabelle di conversione create negli esercizi
- Domande su complemento a 2 o conversioni

GLOSSARIO TECNICO

- **Bit**: Cifra binaria (0 o 1), Binary Digit
- **Byte**: 8 bit, unità base di memoria
- **Nibble**: 4 bit, mezza byte
- **Word**: Parola, dipende dall'architettura (16/32/64 bit)
- **MSB**: Most Significant Bit, bit più significativo (a sinistra)
- **LSB**: Least Significant Bit, bit meno significativo (a destra)
- **Base/Radix**: Numero di cifre in un sistema posizionale

- **Complemento a 2**: Metodo standard per rappresentare negativi
- **Overflow**: Risultato troppo grande per essere rappresentato
- **Underflow**: Risultato troppo piccolo (negativo) per essere rappresentato
- **Signed**: Con segno (positivo/negativo)
- **Unsigned**: Senza segno (solo positivi)
- **Range**: Intervallo di valori rappresentabili

Ottimo lavoro! Ora sai come i computer "pensano" internamente! 🎯

Nella prossima lezione vedremo come vengono rappresentati numeri con virgola e testo.izio 2: Conversioni con Ottale ed Esadecimale (20 min)

2.1 Converti in decimale:

1. 75_8

Soluzione:
$75_8 = 7\times8^1 + 5\times8^0$
$= 7\times8 + 5\times1$
$= 56 + 5$
$= 61_{10}$

1. $2F_{16}$

Soluzione:
$2F_{16} = 2\times16^1 + F\times16^0$
$= 2\times16 + 15\times1$
$= 32 + 15$
$= 47_{10}$

1. $1A0_{16}$

Soluzione:
$1A0_{16} = 1\times16^2 + A\times16^1 + 0\times16^0$
$= 1\times256 + 10\times16 + 0$

$= 256 + 160$

$= 416_{10}$

2.2 Converti 11010110_2 in ottale ed esadecimale:

Soluzione Ottale:
Raggruppa da destra in gruppi di 3:
011 010 110
3 2 6

$11010110_2 = 326_8$

Soluzione Esadecimale:
Raggruppa in gruppi di 4:
1101 0110
D 6

$11010110_2 = D6_{16}$

2.3 Converti $3E8_{16}$ in binario:

Soluzione:
3 E 8
0011 1110 1000

$3E8_{16} = 001111101000_2$

Eserc